



Name of the module: Custom Signal Conditioning
Location in GIT repository: vspa_common\vspa-lib\sigcond

Type of code:

- 10 VCPU kernel functions

VCPU Function API:

```
extern void <customSigCondFnName>(  
    vspa_complex_fixed16 const* pIn,    // Input data pointer  
    vspa_complex_fixed16* pOut,         // Output data address  
    structSigCondParams* pConfig        // Pointer to configuration struct  
);
```

The functions supported are (<customSigCondFnName>)

1. customsigcond_ddc1x_N2560_4t
2. customsigcond_ddc1x_N2560_5t
3. customsigcond_ddc2x_N2560_4t
4. customsigcond_ddc2x_N2560_5t
5. customsigcond_ddc1x_N1152_4t
6. customsigcond_ddc1x_N1152_5t
7. customsigcond_ddc2x_N640_4t
8. customsigcond_ddc2x_N640_5t
9. customsigcond_ddc4x_N640_4t
10. customsigcond_ddc4x_N640_5t
11. customsigcond_ddc8x_N640_4t
12. customsigcond_ddc8x_N640_5t
13. customsigcond_ddc8x_N256_4t
14. customsigcond_ddc8x_N256_5t

Description:

These set of kernels implements the signal conditioning and digital down converter (DDC) chain as shown in Figs 1-4 for 1 input channel. The kernels “customsigcond_ddc1x_N<N>_<nT>t”, “customsigcond_ddc2x_N<N>_<nT>t”, “customsigcond_ddc4x_N<N>_<nT>t”, and, “customsigcond_ddc8x_N<N>_<nT>t” convert an input stream at 160 Msps to 160, 80, 40, 20 Msps respectively. The kernels work on an input buffer of “N” samples and produce an output buffer of N/2, N/4, N/8 respectively. The label <N> in the kernel name refers to the supported size, N, for the input buffer. The label <nT> refers to the number of taps for the fractional delay filter in the I/Q imbalance compensation stage.

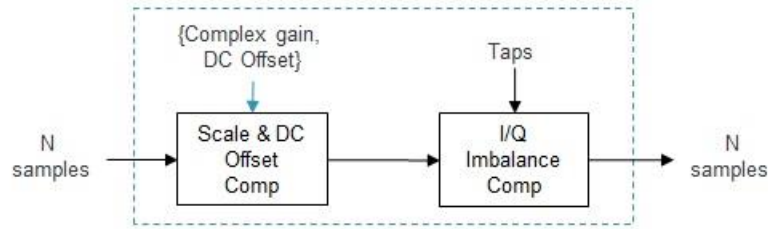


Fig 1: Custom signal conditioning chain for 160MHz

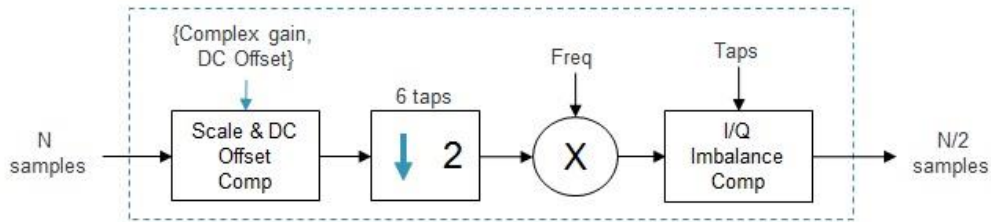


Fig 2: Custom signal conditioning chain for 80MHz

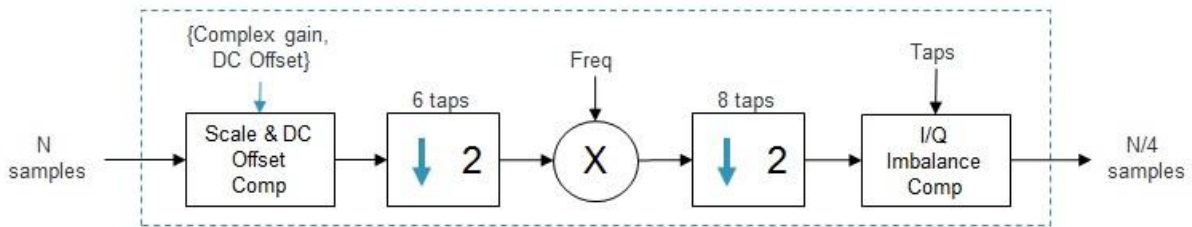


Fig 3: Custom signal conditioning chain for 40MHz

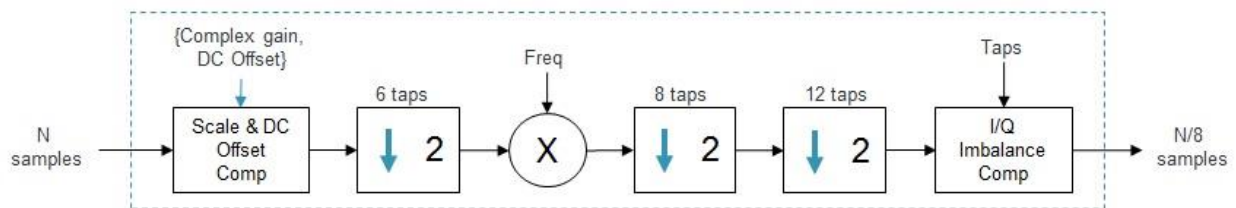


Fig 4: Custom signal conditioning chain for 20MHz

The 80, 40, 20 Mps functions include 1, 2, 3 stages of filtering and decimate-by-2 stages, respectively. In addition to the decimation stages, all functions include a DC offset remover and a complex gain application block operating at 160 Mps, a mixer stage operating at 80 Mps (for 80Mps or lower rate functions), and, an I/Q imbalance compensation block operating at the output rate of the respective function.



The I/Q imbalance compensation block is implemented as shown in Fig 5. The FIR filter $\mathbf{h}_1$ is assumed to have 4 or 5 taps.

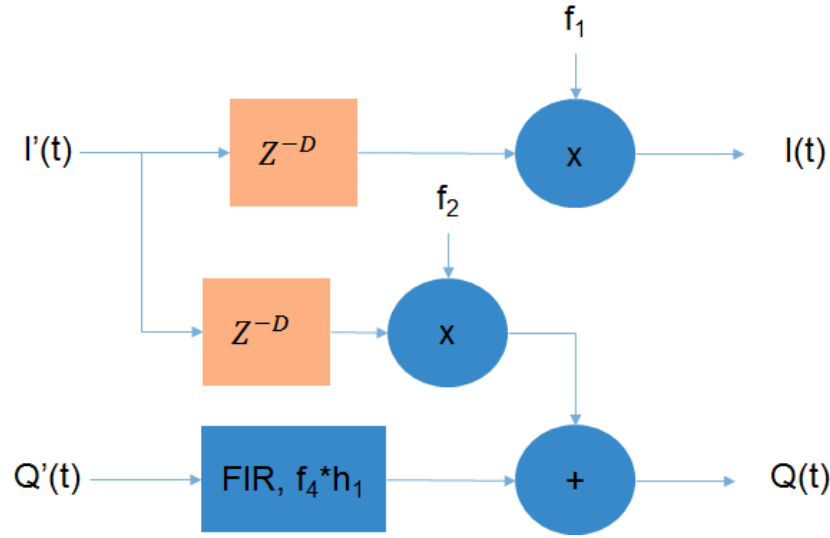


Fig 5: I/Q Imbalance Compensation Block

The I/Q imbalance compensation is based on the following impairment model.

$$\begin{bmatrix} I'(t) \\ Q'(t) \end{bmatrix} = \begin{bmatrix} \alpha & 0 \\ \sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} I(t) \\ Q(t - \tau) \end{bmatrix}$$

Here $\{I(t), Q(t)\}$ are real and imaginary parts of the original input signal, and $\{I'(t), Q'(t)\}$ are real and imaginary parts of the impaired signal. The symbol “ τ ” is a fractional (positive or negative) delay element. The I/Q imbalance compensation module shown in Fig 5 applies the following correction to the impaired stream.

$$I(t) = f_1 * I'(t - D)$$

$$Q(t) = f_2 I'(t - D) + f_4 \sum_{k=0}^{L-1} h_1(k) * Q'(t - k)$$

Here D is a delay derived based on the delay, D_F , of FIR filter $\mathbf{h}_1$ (of length L) adjusted for positive or negative “ τ ”. The filter $\mathbf{h}_1$ is designed to shift the input signal by $(D_F + \delta)$. The parameters are derived as follows:

If $\tau > 0$:

$$\delta = 1 - \tau$$

$$D = D_F - 1$$

Elseif $\tau \leq 0$:

$$\delta = |\tau|$$

$$D = D_F$$

The parameters f_1, f_2, f_4 are derived as



$$\begin{aligned}f_1 &= 1/\alpha \\f_2 &= -\tan(\varphi)/\alpha \\f_4 &= \sec(\varphi)\end{aligned}$$

The kernel maintains persistent states corresponding to the input stages for each decimation-by-2 stage (3 stages for 20 Msps version, 2 stages for 40 Msps version and 1 for 80 Msps version).

The details of the function API are as following.

Explicit arguments

pIn: This is pointer to buffer containing input samples for the current channel at 160 Msps. The samples are expected to be complex and in half-fixed precision. The pointer can be any word (32-bit) aligned address and the buffers can be circular. N+32 valid input samples per channel are expected.

The input pointer needs to be set by the caller to point to the first input sample (not including the filter delay). The caller has to make sure there are at least “d” valid samples prior (in the circular buffer sense) to the first input sample, where d is the composite filter delay for the bandwidth.

pOut: This is pointer to buffer containing the final output samples for the current channels at 80/40/20 Msps. The samples are complex and in half-fixed precision. This pointer must be an integer multiple of the size of a DMEM line. N/2, N/4, or N/8 samples per channel are written per function call for 80, 40, 20 Msps cases respectively.

pConfig: This is pointer to a struct of type *structSigCondParams* that specifies configuration parameters for the signal conditioning chain corresponding to the channel being processed. This pointer must be an integer multiple of the size of a DMEM line. The caller must allocate and maintain one instance of this struct per channel. The structure is defined as:

```
typedef struct {
    vspa_complex_fixed16 filtState[96];
    vspa_complex_fixed16* inCircBuffBase;
    vspa_complex_float32 rxGain;
    vspa_complex_float32 rxDCOffset;
    unsigned int mxrFreq;
    unsigned int mxrPhase;
    float IQImb_ftaps [12];
    unsigned int IQImp_delay;
} structSigCondParams;
```

Each field of the struct is defined below.

- *filtState*: This is an array of type “vspa_complex_fixed16” and size 64 that is required to maintain intermediate states of the decimation filters for the channel. This array is maintained by the kernel. This array should be cleared (zeros) by the caller on initialization/reset.



- *inCircBuffBase*: This is pointer to the base of the circular buffer used for storing input samples for the channel. The size of the circular buffer is defined by the macro “CUSTOMSIGCOND_INPUTCIRCBUFFERSIZE_N<N>” in the header file. If circular buffering is not used, this pointer and the size macro need to be set to zeros.
- *rxGain*: This parameter specifies the complex gain to be applied to the received signal for the channel at the input rate. This is a complex single precision floating point number. This value needs to be set by the caller.
- *rxDCOffset*: This parameter specifies the complex DC offset to be applied to the received signal for the channel at the input rate. This is a complex single precision floating point number. This value needs to be set by the caller.
- *mxrFreq*: This defines the mixer frequency for the channel. This is a 32 bit unsigned integer that denotes the numerically controlled oscillator (NCO) frequency in 1’s complement form. This value needs to be set by the caller.
- *mxrPhase*: This is a persistent unsigned integer that is used to preserve the phase continuity of the mixer for the channel across consecutive calls. This is maintained by the kernel. This value can optionally be cleared on initialization/reset to guarantee deterministic phase progression.
- *IQImb_ftaps*: This is an array of single precision real coefficients of size 10 for I/Q imbalance compensation for the channel. The taps are arranged in memory as shown below (arranged in ascending order of array index from left to right):

For 4 taps case:

$$[0, f_2, h_2(3), h'_1(3), h_2(2), h'_1(2), h_2(1), h'_1(1), h_2(0), h'_1(0), 0, 0]$$

Here, for $k = 0, 1, 2, 3$

$$h'_1(k) = f_4 h_1(k)$$

$$h_2(k) = \begin{cases} 0 & \text{if } k \neq D \\ f_1 & \text{if } k = D \end{cases}$$

The symbols $f_1, f_2, f_4, h_1(k)$ and D are coefficients and delay as defined earlier.

The FIR filter taps are applied to the imaginary input stream $q'(n)$ as shown below.

$$q(n) = h'_1(0)*q'(n) + h'_1(1)*q'(n-1) + h'_1(2)*q'(n-2) + h'_1(3)*q'(n-3)$$

For 5 taps case:

$$[0, f_2, h_2(4), h'_1(4), h_2(3), h'_1(3), h_2(2), h'_1(2), h_2(1), h'_1(1), h_2(0), h'_1(0)]$$



Here, for $k = 0, 1, 2, 3, 4$

$$h'_1(k) = f_4 h_1(k)$$

$$h_2(k) = \begin{cases} 0 & \text{if } k \neq D \\ f_1 & \text{if } k = D \end{cases}$$

The symbols $f_1, f_2, f_4, h_1(k)$ and D are coefficients and delay as defined earlier.

The FIR filter taps are applied to the imaginary input stream $q'(n)$ as shown below.

$$q(n) = h'_1(0)*q'(n) + h'_1(1)*q'(n-1) + h'_1(2)*q'(n-2) + h'_1(3)*q'(n-3) + h'_1(4)*q'(n-4)$$

This array needs to be populated by the caller and should entirely be contained within 1 DMEM line.

- *IQImp_delay*: Integer specifying the delay D for the integer path of the I/Q imbalance compensation block. Given D as described earlier, this value should be set to

$$IQImp_delay = 64 - D*2 - 1$$

Implicit arguments

customsigcond_FilterTaps: This is an array of type “vspa_real_fixed16” that is defined in the header file but needs to be allocated by the caller. The array should hold (6+8+12) real taps for the 3 DDC decimation filtering stages in half-fixed precision and in the order as shown below (in ascending order of array index starting from 0 on the left)

$$[f_1(5) f_1(4) \dots f_1(0) f_2(7) f_2(6) \dots f_2(0) f_3(11) f_3(10) \dots f_3(0)]$$

Here, $f_i(k)$ refers to the tap for the i^{th} decimation filtering stage as applied to input data sample $x(n-k)$ assuming a filter (with L taps) model as shown below.

$$y(n) = f(0)*x(n) + f(1)*x(n-1) + \dots + f(L-1)*x(n-L+1)$$

The entire array should be within a DMEM line and the first tap should be aligned to a 32-bit word boundary. This buffer should be initialized with the appropriate taps prior to calling the function. These taps are common for all channels.

customsigcond_ScratchMem: This defines a scratch memory buffer that is defined in the header file but needs to be allocated by the caller. A kernel processing “ N ” input samples requires $(N+32)$ words (32-bit) of scratch memory.



Latency:

The latency of the signal conditioning block is categorized below in units of samples at the input rate, 160Msps. Let R be the decimation rate. Then $R = \{1, 2, 4, 8\}$ for $\{160, 80, 40, 20\}$ Msps output rates respectively. Then the latency of the first output sample (pOut[0]) with respect to the first input sample (pIn[0]) is given by d_{160} calculated as shown below.

$$d_{160}(R) = (2 \times R + l_{20} \times 8 \times I_{R \geq 8} + l_{40} \times 4 \times I_{R \geq 4} + l_{80} \times 2 \times I_{R \geq 2}) - (4 \times I_{R \geq 8} + 2 \times I_{R \geq 4} + 1 \times I_{R \geq 2}) \times 1.5$$

Here, I_{cond} , is the indicator function given by

$$I_{cond} = \begin{cases} 1, & \text{if } cond = true \\ 0, & \text{otherwise} \end{cases}$$

The stage latencies l_{20} , l_{40} , l_{80} are given by:

$$l_{20} = \text{ceil} \left\{ \left(32 - \frac{L_3}{2} + 1 \right) / 2 \right\}$$

$$l_{40} = \text{ceil} \left\{ \left(32 - \frac{L_2}{2} + 1 \right) / 2 \right\}$$

$$l_{80} = \text{ceil} \left\{ \left(\frac{L_1}{2} \right) / 2 \right\}$$

L_1 , L_2 , L_3 are the number of taps in the decimation filters for stages 1, 2 and 3 respectively. In the current implementation, they are set to $L_1=6$, $L_2=8$, $L_3=12$.

The above formula is only applicable when L_1 , L_2 , L_3 are even and the filters are symmetric. The latency in microseconds (us) is given by $d_{160}/160$. The numbers are summarized in the table below:

Output Rate (Msps)	Latency (in samples @ 160 Msps)	Latency (us)
160	2	0.0125
80	6.5	0.0406
40	67.5	0.4219
20	181.5	1.1344